

Integrity and Security

This chapter presents several types of integrity constraints, including domain constraints, referential integrity constraints, assertions and triggers, as well as security and authorization. Referential integrity constraints, and domain constraints are an important aspect of the specification of a relational database design. Assertions are seeing increasing use. Triggers are widely used, although each database supports its own syntax and semantics for triggers; triggers were standardized as part of SQL:1999, and we can expect databases to provide support for SQL:1999 triggers.

Functional dependencies are now taught as part of normalization instead of being part of the integrity constraints chapter as they were in the 3rd edition. The reason for the change is that they are used almost exclusively in database design, and no database system to our knowledge supports functional dependencies as integrity constraints. Covering them in the context of normalization helps motivate students to spend the effort to understand the intricacies of reasoning with functional dependencies.

Security is a major topic in its own right. Since any system is only as secure as its weakest component, a system builder must consider all aspects of security. This chapter focuses only on those security issues that are specific to databases. In an advanced course, this material can be supplemented by discussion of security issues in operating systems and in distributed systems.

Changes from 3rd edition:

Trigger coverage is now based on the SQL:1999 standard. At the time of publication of the 3rd edition, triggers had not been standardized. The notion of roles for authorization has been introduced in this edition, now that it is a part of the SQL:1999 standard. Coverage of encryption has been updated to cover recent developments.

Exercises

- 6.1 Complete the SQL DDL definition of the bank database of Figure 6.2 to include the relations *loan* and *borrower*.

Answer:

```
create table loan
  (loan-number   char(10),
   branch-name   char(15),
   amount        integer,
   primary key (loan-number),
   foreign key (branch-name) references branch)
```

```
create table borrower
  (customer-name char(20),
   loan-number   char(10),
   primary key (customer-name, loan-number),
   foreign key (customer-name) references customer,
   foreign key (loan-number) references loan)
```

Declaring the pair *customer-name*, *loan-number* of relation *borrower* as primary key ensures that the relation does not contain duplicates.

- 6.2 Consider the following relational database:

```
employee (employee-name, street, city)
works (employee-name, company-name, salary)
company (company-name, city)
manages (employee-name, manager-name)
```

Give an SQL DDL definition of this database. Identify referential-integrity constraints that should hold, and include them in the DDL definition.

Answer:

```
create table employee
  (person-name   char(20),
   street         char(30),
   city           char(30),
   primary key (person-name )
```

```

create table works
  (person-name   char(20),
   company-name char(15),
   salary        integer,
   primary key (person-name),
   foreign key (person-name) references employee,
   foreign key (company-name) references company)

```

```

create table company
  (company-name char(15),
   city          char(30),
   primary key (company-name))

```

```

create table manages
  (person-name   char(20),
   manager-name char(20),
   primary key (person-name),
   foreign key (person-name) references employee,
   foreign key (manager-name) references employee)

```

Note that alternative datatypes are possible. Other choices for **not null** attributes may be acceptable.

- 6.3 Referential-integrity constraints as defined in this chapter involve exactly two relations. Consider a database that includes the following relations:

```

salaried-worker (name, office, phone, salary)
hourly-worker  (name, hourly-wage)
address (name, street, city)

```

Suppose that we wish to require that every name that appears in *address* appear in either *salaried-worker* or *hourly-worker*, but not necessarily in both.

- a. Propose a syntax for expressing such constraints.
- b. Discuss the actions that the system must take to enforce a constraint of this form.

Answer:

- a. For simplicity, we present a variant of the SQL syntax. As part of the **create table** expression for *address* we include

```

foreign key (name) references salaried-worker or hourly-worker

```

- b. To enforce this constraint, whenever a tuple is inserted into the *address* relation, a lookup on the *name* value must be made on the *salaried-worker* relation and (if that lookup failed) on the *hourly-worker* relation (or vice-versa).

- 6.4 SQL allows a foreign-key dependency to refer to the same relation, as in the following example:

```
create table manager
(employee-name char(20),
 manager-name char(20),
 primary key employee-name,
 foreign key (manager-name) references manager
 on delete cascade )
```

Here, *employee-name* is a key to the table *manager*, meaning that each employee has at most one manager. The foreign-key clause requires that every manager also be an employee. Explain exactly what happens when a tuple in the relation *manager* is deleted.

Answer: The tuples of all employees of the manager, at all levels, get deleted as well! This happens in a series of steps. The initial deletion will trigger deletion of all the tuples corresponding to direct employees of the manager. These deletions will in turn cause deletions of second level employee tuples, and so on, till all direct and indirect employee tuples are deleted.

- 6.5 Suppose there are two relations *r* and *s*, such that the foreign key *B* of *r* references the primary key *A* of *s*. Describe how the trigger mechanism can be used to implement the **on delete cascade** option, when a tuple is deleted from *s*.

Answer: We define triggers for each relation whose primary-key is referred to by the foreign-key of some other relation. The trigger would be activated whenever a tuple is deleted from the referred-to relation. The action performed by the trigger would be to visit all the referring relations, and delete all the tuples in them whose foreign-key attribute value is the same as the primary-key attribute value of the deleted tuple in the referred-to relation. These set of triggers will take care of the **on delete cascade** operation.

- 6.6 Write an assertion for the bank database to ensure that the assets value for the Perryridge branch is equal to the sum of all the amounts lent by the Perryridge branch.

Answer: The assertion-name is arbitrary. We have chosen the name *perry*. Note that since the assertion applies only to the Perryridge branch we must restrict attention to only the Perryridge tuple of the *branch* relation rather than writing a constraint on the entire relation.

```
create assertion perry check
(not exists (select *
from branch
where branch-name = 'Perryridge' and
assets ≠ (select sum (amount)
from loan
where branch-name = 'Perryridge'))))
```

- 6.7 Write an SQL trigger to carry out the following action: On **delete** of an account, for each owner of the account, check if the owner has any remaining accounts, and if she does not, delete her from the *depositor* relation.

Answer:

```
create trigger check-delete-trigger after delete on account
referencing old row as orow
for each row
delete from depositor
where depositor.customer-name not in
( select customer-name from depositor
  where account-number <> orow.account-number )
end
```

- 6.8 Consider a view *branch-cust* defined as follows:

```
create view branch-cust as
select branch-name, customer-name
from depositor, account
where depositor.account-number = account.account-number
```

Suppose that the view is *materialized*, that is, the view is computed and stored. Write active rules to *maintain* the view, that is, to keep it up to date on insertions to and deletions from *depositor* or *account*. Do not bother about updates.

Answer: For inserting into the materialized view *branch-cust* we must set a database trigger on an insert into *depositor* and *account*. We assume that the database system uses *immediate* binding for rule execution. Further, assume that the current version of a relation is denoted by the relation name itself, while the set of newly inserted tuples is denoted by qualifying the relation name with the prefix – **inserted**.

The active rules for this insertion are given below –

```
define trigger insert_into_branch-cust_via_depositor
after insert on depositor
referencing new table as inserted for each statement
insert into branch-cust
select branch-name, customer-name
from inserted, account
where inserted.account-number = account.account-number
```

```
define trigger insert_into_branch-cust_via_account
after insert on account
referencing new table as inserted for each statement
insert into branch-cust
select branch-name, customer-name
from depositor, inserted
where depositor.account-number = inserted.account-number
```

Note that if the execution binding was *deferred* (instead of immediate), then the result of the join of the set of new tuples of *account* with the set of new tuples of *depositor* would have been inserted by *both* active rules, leading to duplication of the corresponding tuples in *branch-cust*.

The deletion of a tuple from *branch-cust* is similar to insertion, except that a deletion from either *depositor* or *account* will cause the natural join of these relations to have a lesser number of tuples. We denote the newly deleted set of tuples by qualifying the relation name with the keyword **deleted**.

```
define trigger delete_from_branch-cust_via_depositor
after delete on depositor
referencing old table as deleted for each statement
delete from branch-cust
    select branch-name, customer-name
    from deleted, account
    where deleted.account-number = account.account-number
```

```
define trigger delete_from_branch-cust_via_account
after delete on account
referencing old table as deleted for each statement
delete from branch-cust
    select branch-name, customer-name
    from depositor, deleted
    where depositor.account-number = deleted.account-number
```

- 6.9 Make a list of security concerns for a bank. For each item on your list, state whether this concern relates to physical security, human security, operating-system security, or database security.

Answer: Let us consider the problem of protecting our sample bank database. Some security measures at each of the four levels are mentioned below -

- a. Physical level - The system from which the relations can be accessed and modified should be placed in a locked, well-guarded, and impregnable room.
- b. Human level - A proper key transfer policy should be enforced for restricting access to the "system room" mentioned above. Passwords for gaining access to the database should be known only to trusted users.
- c. Operating System level - Login passwords should be difficult to guess and they should be changed regularly. No user should be able to gain unauthorized access to the system due to a software bug in the operating system.
- d. Database System level - The users should be authorized access only to relevant parts of the database. For example, a bank teller should be allowed to modify values for the customer's balance, but not for her own salary.

- 6.10 Using the relations of our sample bank database, write an SQL expression to define the following views:

- a. A view containing the account numbers and customer names (but not the balances) for all accounts at the Deer Park branch.

- b. A view containing the names and addresses of all customers who have an account with the bank, but do not have a loan.
- c. A view containing the name and average account balance of every customer of the Rock Ridge branch.

Answer:

a.

```
create view deer-park as
select account-number, customer-name
from depositor, account
where branch-name = 'Deer Park' and
       depositor.account-number = account.account-number
```

b.

```
create view no-debt as
select * from customer
where customer-name in
      (select customer-name
       from depositor)
minus
      (select customer-name
       from borrower)
```

c.

```
create view avg-bal as
select customer-name, avg(balance)
from depositor, account
where depositor.account-number = account.account-number
       and branch-name = 'Rock Ridge'
group by customer-name
```

- 6.11 For each of the views that you defined in Exercise 6.10, explain how updates would be performed (if they should be allowed at all). *Hint:* See the discussion of views in Chapter 3.

Answer: To insert (*account-number, name*) into the view *deer-park* we insert the tuple (Deer Park, *account-number, null*) into the *account* relation and the tuple (*name, account-number*) into the *depositor* relation.

Updates to the views *no-debt* and *avg-bal* present serious problems. If we insert into the *no-debt* view, the system must reject the insertion if the customer has a loan. The overhead of updating through this view is so high that most systems would disallow update. The *avg-bal* view cannot be updated since the result of an aggregate operation depends on several tuples, not just one.

- 6.12 In Chapter 3, we described the use of views to simplify access to the database by users who need to see only part of the database. In this chapter, we described the use of views as a security mechanism. Do these two purposes for views ever conflict? Explain your answer.

Answer: Usually, a well-designed view and security mechanism can avoid con-

flicts between ease of access and security. However, as the following example shows, the two purposes do conflict in case the mechanisms are not designed carefully.

Suppose we have a database of employee data and a user whose view involves employee data for employees earning less than \$10,000. If this user inserts employee Jones, whose salary is \$9,000, but accidentally enters \$90,000, several existing database systems will accept this update as a valid update through a view. However, the user will be denied access to delete this erroneous tuple by the security mechanism.

- 6.13** What is the purpose of having separate categories for index authorization and resource authorization?

Answer: Index and resource authorization should be special categories to allow certain users to create relations (and the indices to operate on them) while preventing these time-consuming and schema-changing operations from being available to many users. Separating index and resource authorization allows a user to build an index on existing relations, say, for optimization purposes, but allows us to deny that user the right to create new relations.

- 6.14** Database systems that store each relation in a separate operating-system file may use the operating system's security and authorization scheme, instead of defining a special scheme themselves. Discuss an advantage and a disadvantage of such an approach.

Answer: Database systems have special requirements which are typically more refined than most operating systems. For example, a single user may have different privileges on different files throughout the system, including changing indices and attributes which file systems typically don't monitor. The advantage of using the operating system's security mechanism is that it simplifies the database system and can be used for simple (read/write) security measures.

- 6.15** What are two advantages of encrypting data stored in the database?

Answer:

- a. Encrypted data allows authorized users to access data without worrying about other users or the system administrator gaining any information.
- b. Encryption of data may simplify or even strengthen other authorization mechanisms. For example, distribution of the cryptographic key amongst only trusted users is both, a simple way to control read access, and an added layer of security above that offered by views.

- 6.16** Perhaps the most important data items in any database system are the passwords that control access to the database. Suggest a scheme for the secure storage of passwords. Be sure that your scheme allows the system to test passwords supplied by users who are attempting to log into the system.

Answer: A scheme for storing passwords would be to encrypt each password, and then use a hash index on the user-id. The user-id can be used to easily access the encrypted password. The password being used in a login attempt is then encrypted and compared with the stored encryption of the correct password. An

advantage of this scheme is that passwords are not stored in clear text and the code for decryption need not even exist!